

Gestion du temps et déclenchement avec délai

Dans Unity, il est souvent nécessaire de gérer des actions qui se produisent après un certain délai. Cela peut être utile pour créer des effets temporisés, des animations différées, ou pour contrôler le flux de jeu. Voici quelques méthodes courantes pour gérer le temps et déclencher des actions avec un délai.

1.1 La classe Time

Unity fournit la classe `Time` qui contient plusieurs propriétés utiles pour gérer le temps dans vos jeux. Voici quelques-unes des plus courantes :

- `Time.deltaTime` : Le temps écoulé entre la dernière et la dernière frame. Utile pour rendre les mouvements indépendants du framerate. Cela permet de s'assurer que les objets se déplacent à la même vitesse, quel que soit le nombre de frames par seconde et peu importe la performance de l'appareil sur lequel le jeu tourne.
- `Time.time` : Le temps écoulé depuis le début du jeu.
- `Time.timeScale` : Permet de modifier la vitesse à laquelle le temps s'écoule dans le jeu. Par exemple, une valeur de 0.5 ralentira le temps de moitié, tandis qu'une valeur de 2 accélérera le temps. Cela peut être utilisé pour créer des effets de ralenti ou d'accélération.

1.1.1 Créer un minuteur

Pour afficher un minuteur dans votre jeu, vous pouvez utiliser une variable pour suivre le temps écoulé et mettre à jour cette variable dans la méthode `Update()`. Voici un exemple simple de minuteur qui compte à rebours à partir de 10 secondes :

Dans cet exemple, nous utilisons un élément `TextMeshPro` pour afficher le temps restant à l'écran. Aussi, nous utilisons `Mathf.Ceil` pour arrondir le temps restant à l'entier supérieur afin d'afficher des secondes complètes et non des fractions de secondes.

```
using UnityEngine;
using TMPro;

public class Jeu:MonoBehaviour
{
    public TMP_Text texteMinuteur; // Référence à l'élément TextMeshPro pour afficher le minuteur
    private float tempsRestant = 10f; // Temps initial en secondes

    void Update()
    {
        if (tempsRestant > 0)
        {
            tempsRestant -= Time.deltaTime; // Réduire le temps restant
            texteMinuteur.text = $"Temps restant: {Mathf.Ceil(tempsRestant)}s";
        }
    }
}
```

```
        else
        {
            texteMinuteur.text = "Temps écoulé!";
        }
    }
}
```

1.2 Déclencher une action avec un délai

La fonction `Invoke` permet de déclencher une méthode après un certain délai. C'est l'équivalent de `setTimeout` en JavaScript. Pour utiliser `Invoke`, vous devez spécifier le nom de la méthode à appeler et le délai en secondes. Attention, vous devez passer le nom de la méthode en tant que chaîne de caractères (attention aux majuscules/minuscules) et la méthode doit être définie dans le même script.

Si vous appelez `Invoke` dans la méthode `Start()`, l'action sera déclenchée après le délai spécifié dès le début du jeu.

Attention, si vous appelez `Invoke` dans la méthode `Update()`, l'action sera reprogrammée à chaque frame, ce qui peut entraîner des comportements inattendus. Il est généralement préférable d'appeler `Invoke` dans `Start()` ou de l'envelopper dans une condition pour éviter cela.

Voici un exemple simple :

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class DelaiExemple : MonoBehaviour
{
    public float pointsVies = 100f;
    bool estMort = false;

    void Start()
    {
        // Appeler la méthode "FonctionAAppelerAuDemarrage" après 3 secondes
        Invoke("FonctionAAppelerAuDemarrage", 3f);
    }

    void FonctionAAppelerAuDemarrage()
    {
        Debug.Log("Le jeu a commencé!");
    }

    void Update()
    {
        // Cela empêche d'appeler Invoke à chaque frame car tout le bloc est dans une condition
        if (!estMort)
        {
            pointsVies -= Time.deltaTime * 10; // Réduire les points de vie au fil du temps
        }
    }
}
```

```
        if (pointsVies <= 0)
        {
            estMort = true; // Empêche d'appeler Invoke à chaque frame
            // Appeler la méthode "Changer" après 3 secondes
            Invoke("RedemarrerJeu", 3f);
        }
    }

    void RedemarrerJeu()
    {
        Debug.Log("Action déclenchée après un délai de 3 secondes!");
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex); // Redémarrer la scène actuelle
    }
}
```

1.3 Déclencher une fonction à répétition

Vous pouvez utiliser `InvokeRepeating` pour appeler une méthode de manière répétée à des intervalles spécifiés. C'est l'équivalent de `setInterval` en JavaScript. Vous devez spécifier le nom de la méthode, le délai initial avant le premier appel, et l'intervalle entre les appels suivants.

```
using UnityEngine;
public class RepetitionExample : MonoBehaviour
{
    void Start()
    {
        // Appeler la méthode "FonctionARepetee" toutes les 2 secondes, après un délai initial de 1 seconde
        InvokeRepeating("FonctionARepetee", 1f, 2f);
    }

    void FonctionARepetee()
    {
        Debug.Log("Cette fonction est appelée toutes les 2 secondes.");
    }
}
```

1.4 Annuler un Invoke ou InvokeRepeating

Vous pouvez annuler un appel programmé avec `Invoke` ou `InvokeRepeating` en utilisant la méthode `CancelInvoke`. Voici comment l'utiliser :

```
using UnityEngine;
public class AnnulerInvokeExample : MonoBehaviour
{
    public int repetitionCount = 0;
```

```
void Start()
{
    // Appeler la méthode "FonctionARepetee" toutes les 2 secondes, après un délai initial de 1 seconde
    InvokeRepeating("FonctionARepetee", 1f, 2f);

}

void FonctionARepetee()
{
    Debug.Log("Cette fonction est appelée toutes les 2 secondes.");
    repetitionCount++;

    if (repetitionCount >= 5)
    {
        AnnulerFonctionARepetee();
    }
}

void AnnulerFonctionARepetee()
{
    CancelInvoke("FonctionARepetee");
    Debug.Log("L'appel répété a été annulé.");
}
}
```

1.5 Utilisation de la classe Time et Invoke pour ralentir le jeu pendant 3 secondes

```
using UnityEngine;
public class RalentirJeuExemple : MonoBehaviour
{
    public bool tempsRalenti = false;

    //Lorsque le personnage entre en collision avec un autre objet
    void OnCollisionEnter2D(Collision2D collision)
    {
        //Éviter de relancer le ralentissement si le temps est déjà ralenti et qu'il y a déjà d'autres
        // collisions
        if(!tempsRalenti){
            tempsRalenti = true;
            // Ralentir le temps à 0.5x la vitesse normale lors d'une collision
            Time.timeScale = 0.5f;
            // Revenir à la vitesse normale après 3 secondes de temps réel
            Invoke("ReinitialiserTemps", 3f);
        }
    }
}
```

```
void ReinitialiserTemps()
{
    tempsRalenti = false;
    Time.timeScale = 1f; // Revenir à la vitesse normale
    Debug.Log("Le temps est revenu à la normale.");
}
}
```

1.6 Pour aller plus loin

Nous verrons la session prochaine qu'il est possible de déclencher une fonction avec délai en utilisant des coroutines, ce qui offre plus de flexibilité pour gérer des séquences d'actions dans le temps et déclencher des fonctions avec paramètres.

Si cela vous intéresse : <https://learn.unity.com/tutorial/coroutines>

[Documentation officielle Unity sur la classe Time](#) [Documentation officielle Unity sur Invoke](#) [Documentation officielle Unity sur InvokeRepeating](#)